

Project Title:

Secure Multi-Tier Network Architecture on AWS

Cloud Computing BSE-VB

Submitted by:

Tooba Shafique (2023-BSE-065)

Seerat Fatima (2023-BSE-060)

Submitted to:

Sir Waqas

Submission Date:

27-01-2026

Contents

1. Repository Link.....	3
2. Executive Summary	3
3. Architecture Design.....	3
4. Terraform Implementation — Part 1 — Terraform Modules (Network, Security, Compute)	5
5. Part 2 — Terraform Execution & Outputs.....	8
6. Part 3 — Root Terraform & Environments	12
7. Part 4 — Ansible Implementation	15
8. Part 5 — Security Testing & Compliance Checklist	21
9. Part 6 — Challenges & Solutions	22
10. Part 7 — Conclusion & Future Improvements	24
11. Appendices	24

1. Repository Link

[GitHub Repository] (Repo link: <https://github.com/toobashafique065/Project4-Secure-MultiTier>)

2. Executive Summary

This project demonstrates the deployment of a **secure three-tier architecture** on AWS using Infrastructure as Code (IaC) and configuration management tools. The solution leverages **Terraform** for provisioning, **Ansible** for server configuration, **Nginx** for reverse proxying, and **Git/GitHub** for version control and collaboration.

The architecture addresses key security concerns:

- Prevents direct SSH access to internal servers
- Enforces least-privilege access via Security Groups
- Isolates the application tier in private subnets
- Implements SSH hardening and centralized log collection

The final outcome is a **fully automated, secure cloud infrastructure** with a bastion host, hardened SSH access, and verified compliance.

3. Architecture Design

3.1 High-Level Network Architecture

The infrastructure follows a secure **three-tier architecture** deployed within an AWS VPC:

- **Web Tier** (Public Subnet): Hosts an Nginx reverse proxy with a public IP. It receives HTTP traffic from Internet users on port 80 and forwards requests to the application tier.
- **Application Tier** (Private Subnet): Hosts Flask application servers running on port 8080. These instances have no public IPs and are only accessible from the web tier or bastion host.
- **Management Tier** (Management Subnet): Contains a bastion host with a public IP. It acts as the single SSH entry point for administrators, allowing secure access to web and app servers via private IPs.

Traffic Flow

- **HTTP:** Internet → Web Tier → Application Tier
- **SSH:** Admin → Bastion → Web/App (via private IPs)

Screenshot:

- project4_architecture_diagram.png

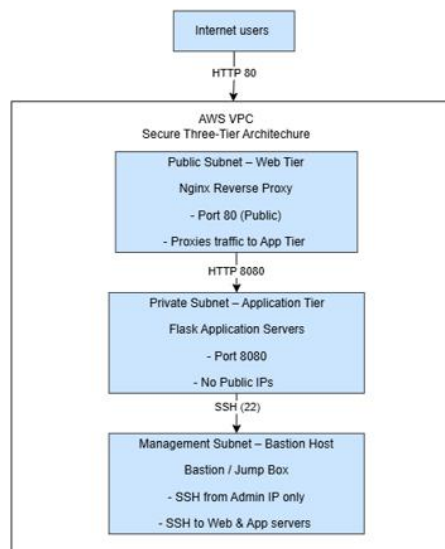


Figure1: architecture_diagram

3.2 Security Architecture Overview

The architecture enforces **defense-in-depth security** through subnet isolation and strict access controls:

- **Network Segmentation:**
 - Public subnet for web tier
 - Private subnet for application tier
 - Management subnet for bastion host
- **Least-Privilege Security Groups:**
 - Web tier allows HTTP (port 80) from the Internet and SSH only from the bastion
 - App tier allows HTTP (port 8080) from the web tier and SSH only from the bastion
 - Bastion allows SSH (port 22) only from a trusted admin IP
- **Bastion Host as Single Entry Point:** All SSH access is routed through the bastion, preventing direct access to internal servers.
- **No Public IPs on Application Servers:** App tier instances are fully isolated from the public Internet, reducing attack surface.

Screenshot:

- project4_security_architecture_diagram.png

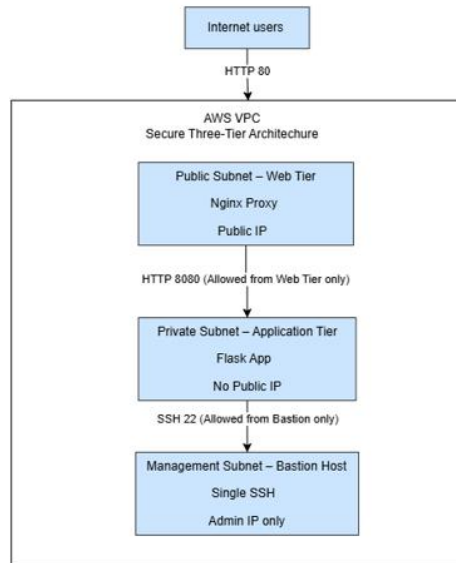


Figure2: security_architecture_diagram

4. Terraform Implementation — Part 1 — Terraform Modules (Network, Security, Compute)

4.1 Repository Structure

The repository is logically organized into separate directories for:

- **Terraform modules** (network, security, compute)
- **Ansible roles and playbooks**
- **Application code**
- **Documentation and logs**

This modular structure improves maintainability, supports reuse across environments, and ensures clarity for future audits and collaboration.

Screenshot:

- project4_part1_repository_structure.png

```

@toobashafique065 /workspaces/Project4-Secure-MultiTier/Project4 (main) $ cat README.md
# Project 4 - Secure Multi-Tier Network

## Repository Structure
Project4-Secure-MultiTier/
├── README.md
├── .gitignore
├── terraform/
│   ├── main.tf
│   ├── variables.tf
│   ├── outputs.tf
│   ├── backend.tf
│   ├── terraform.tfvars.example
│   └── environments/
│       ├── dev.tfvars
│       ├── staging.tfvars
│       └── production.tfvars
│   └── modules/
│       ├── network/
│       ├── security/
│       └── compute/
├── ansible/
│   ├── ansible.cfg
│   ├── inventory/
│   ├── playbooks/
│   ├── roles/
│   └── group_vars/
├── app/
│   └── api/
├── docs/
│   ├── architecture.md
│   ├── security-architecture.md
│   ├── security-checklist.md
│   ├── troubleshooting.md
│   └── logs/

```

- project4_part1_initial_commit.png

```

@toobashafique065 →/workspaces/Project4-Secure-MultiTier/Project4 (main) $ git commit -m "Part 1: Repo
structure, branching, and gitignore setup"
[main 6026a69] Part 1: Repo structure, branching, and gitignore setup
34 files changed, 76 insertions(+)
create mode 100644 Project4/.gitignore
create mode 100644 Project4/README.md
create mode 100644 Project4/ansible/ansible.cfg
create mode 100644 Project4/ansible/group_vars/all.yml
create mode 100644 Project4/ansible/inventory/dev_aws_ec2.yml
create mode 100644 Project4/ansible/inventory/production_aws_ec2.yml
create mode 100644 Project4/ansible/inventory/staging_aws_ec2.yml
create mode 100644 Project4/ansible/playbooks/collect-logs.yml
create mode 100644 Project4/ansible/playbooks/configure-nginx-proxy.yml
create mode 100644 Project4/ansible/playbooks/harden-servers.yml
create mode 100644 Project4/ansible/playbooks/verify-security.yml
create mode 100644 Project4/ansible/roles/app/tasks/main.yml
create mode 100644 Project4/ansible/roles/hardening/tasks/main.yml
create mode 100644 Project4/ansible/roles/hardening/templates/sshd_config.j2
create mode 100644 Project4/ansible/roles/nginx-reverse-proxy/tasks/main.yml
create mode 100644 Project4/ansible/roles/nginx-reverse-proxy/templates/nginx.conf.j2

```

4.2 Git Ignore & Security

The .gitignore file is configured to exclude sensitive and auto-generated files such as:

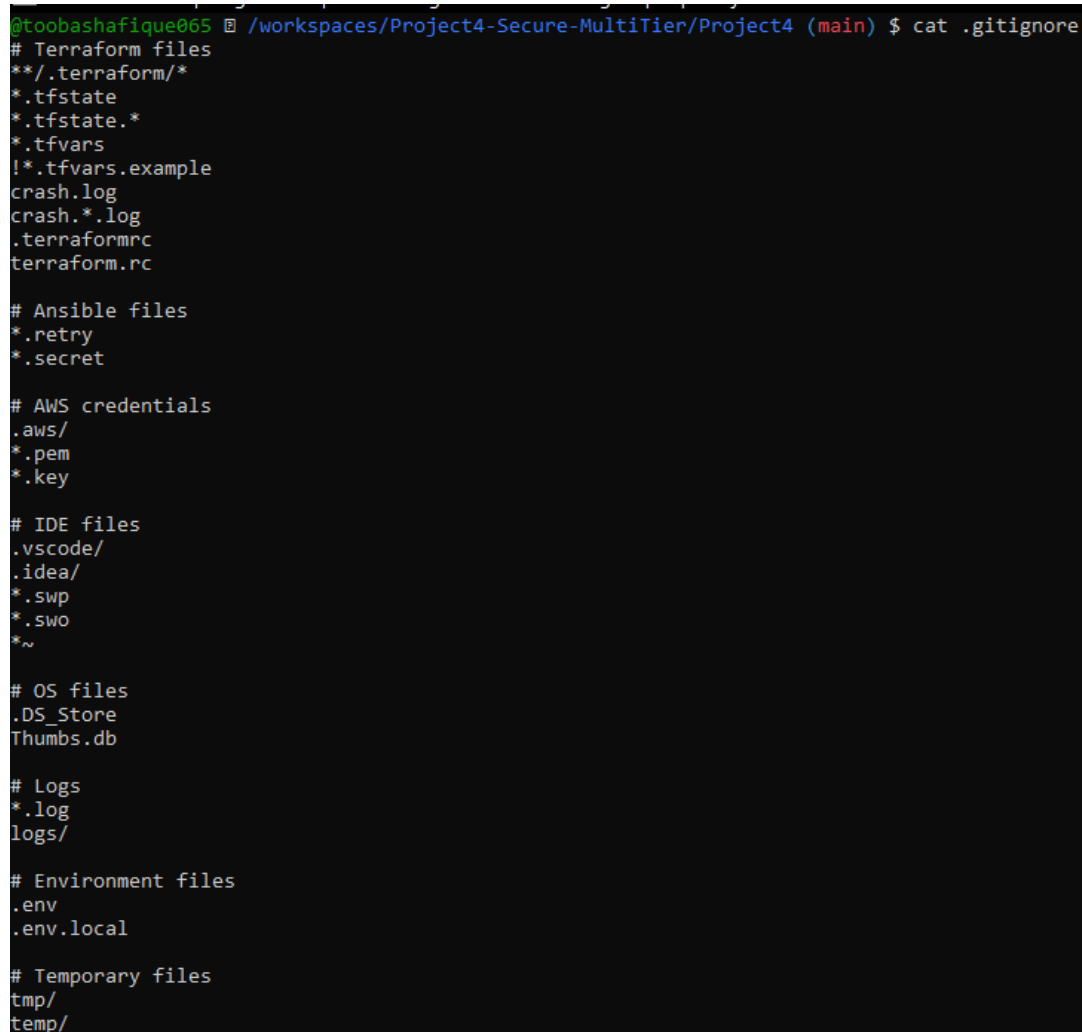
- SSH keys
- Terraform state files (*.tfstate)
- AWS credentials

- Log files and temporary artifacts

This prevents accidental exposure of secrets and ensures a clean, secure version history.

Screenshots:

- project4_part1_gitignore.png



```
@toobashafique065 /workspaces/Project4-Secure-MultiTier/Project4 (main) $ cat .gitignore
# Terraform files
**/.terraform/*
*.tfstate
*.tfstate.*
*.tfvars
!*.tfvars.example
crash.log
crash.*.log
.terraformrc
terraform.rc

# Ansible files
*.retry
*.secret

# AWS credentials
.aws/
*.pem
*.key

# IDE files
.vscode/
.idea/
*.swp
*.swo
*~

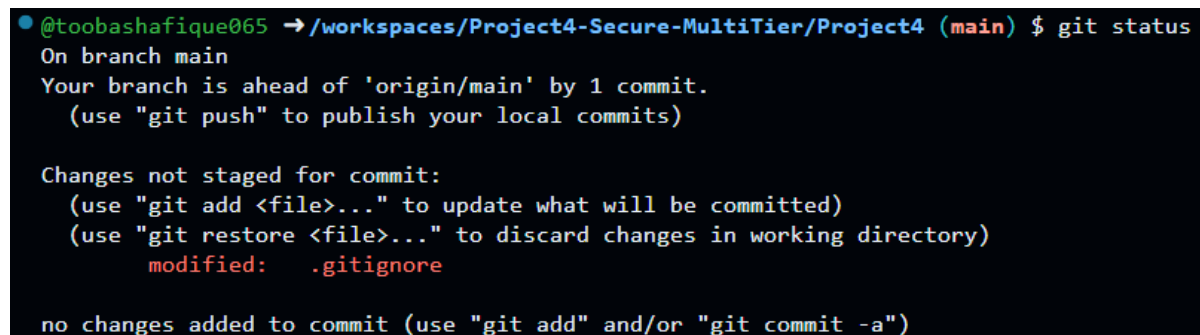
# OS files
.DS_Store
Thumbs.db

# Logs
*.log
logs/

# Environment files
.env
.env.local

# Temporary files
tmp/
temp/
```

- project4_part1_git_status_clean.png



```
@toobashafique065 →/workspaces/Project4-Secure-MultiTier/Project4 (main) $ git status
On branch main
Your branch is ahead of 'origin/main' by 1 commit.
  (use "git push" to publish your local commits)

Changes not staged for commit:
  (use "git add <file>..." to update what will be committed)
  (use "git restore <file>..." to discard changes in working directory)
        modified:   .gitignore

no changes added to commit (use "git add" and/or "git commit -a")
```

4.3 Git Branching Strategy

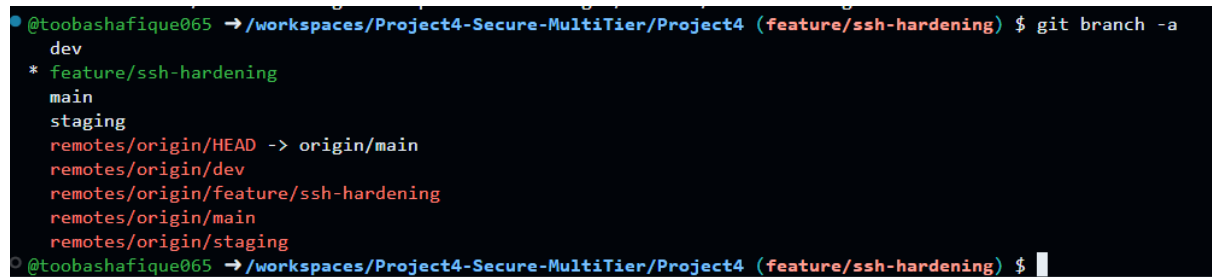
The project uses a structured Git branching model:

- main → stable production-ready code
- staging → pre-production testing
- dev → active development
- feature/* → isolated feature branches for modular changes

This strategy supports parallel development, safe testing, and controlled releases.

Screenshots:

- project4_part1_git_branches.png



```
@toobashafique065 → /workspaces/Project4-Secure-MultiTier/Project4 (feature/ssh-hardening) $ git branch -a
dev
* feature/ssh-hardening
main
staging
remotes/origin/HEAD -> origin/main
remotes/origin/dev
remotes/origin/feature/ssh-hardening
remotes/origin/main
remotes/origin/staging
@toobashafique065 → /workspaces/Project4-Secure-MultiTier/Project4 (feature/ssh-hardening) $
```

- project4_part1_branching_diagram.png



5. Part 2 — Terraform Execution & Outputs

5.1 VPC & Subnet Design

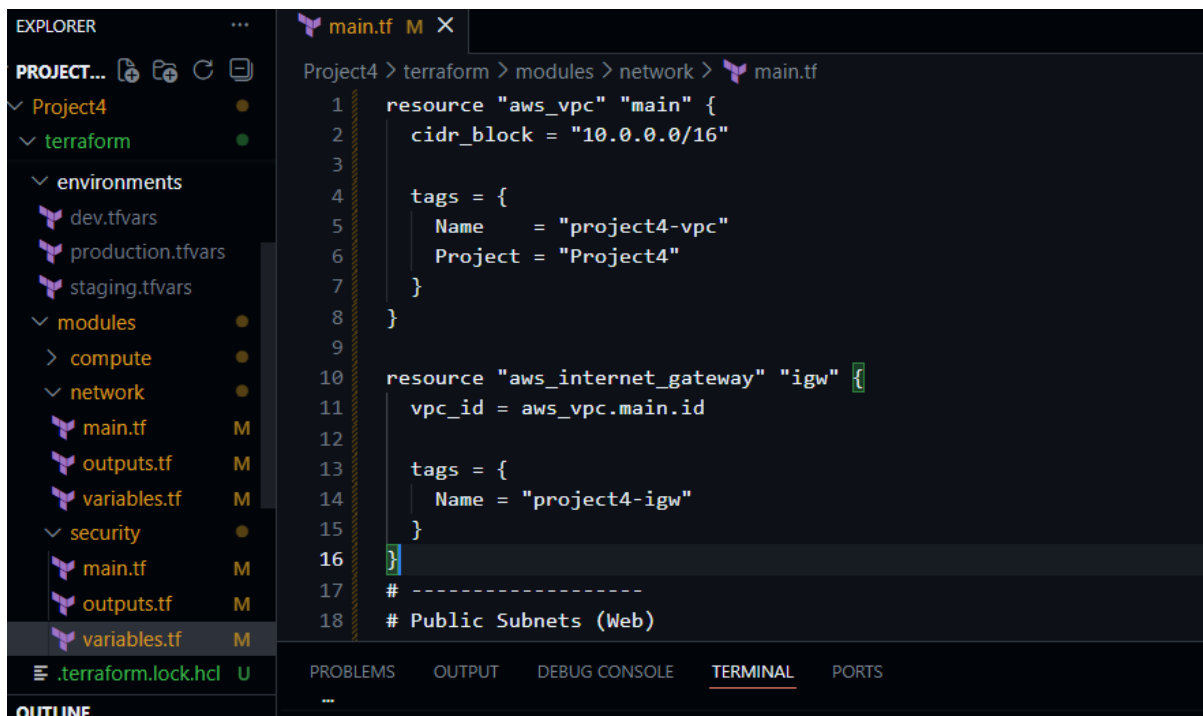
The VPC is defined with a CIDR block (e.g., 10.0.0.0/16) and segmented into:

- **Public subnet** for web tier
- **Private subnet** for application tier
- **Management subnet** for bastion host

An Internet Gateway is attached to the public subnet, and route tables are configured to control traffic flow.

Screenshots:

- project4_part2_network_main.png



The screenshot shows the VS Code interface with the Explorer panel on the left displaying the project structure. The main editor shows the `main.tf` file in the `Project4 > terraform > modules > network` directory. The code defines an `aws_vpc` resource and an `aws_internet_gateway` resource.

```
1 resource "aws_vpc" "main" {
2   cidr_block = "10.0.0.0/16"
3
4   tags = {
5     Name      = "project4-vpc"
6     Project   = "Project4"
7   }
8 }
9
10 resource "aws_internet_gateway" "igw" {
11   vpc_id = aws_vpc.main.id
12
13   tags = {
14     Name = "project4-igw"
15   }
16 }
17 # -----
18 # Public Subnets (Web)
```

- `project4_part2_network_outputs.png`



The screenshot shows the VS Code interface with the Explorer panel on the left. The main editor shows the `outputs.tf` file in the `Project4 > terraform > modules > network` directory. The code defines three output blocks: `vpc_id`, `public_subnet_ids`, and `private_subnet_ids`.

```
1 output "vpc_id" {
2   value = aws_vpc.main.id
3 }
4
5 output "public_subnet_ids" {
6   value = [
7     aws_subnet.public_a.id,
8     aws_subnet.public_b.id
9   ]
10 }
11
12 output "private_subnet_ids" {
13   value = [
14     aws_subnet.private_a.id,
15     aws_subnet.private_b.id
16   ]
17 }
18
```

5.2 Security Groups (Least Privilege)

Security Groups enforce strict access rules:

- **Web SG:** Allows HTTP (80) from Internet, SSH only from bastion

- **App SG:** Allows HTTP (8080) from web tier, SSH only from bastion
- **Bastion SG:** Allows SSH (22) only from admin IP

Security Group-to-Group communication ensures internal access without exposing private servers.

Screenshots:

- project4_part2_security_main.png

```

1 resource "aws_security_group" "bastion_sg" {
2     name           = "bastion-sg"
3     description    = "Allow SSH from admin IP"
4     vpc_id         = var.vpc_id
5
6     ingress {
7         from_port   = 22
8         to_port     = 22
9         protocol    = "tcp"
10        cidr_blocks = [var.admin_ip]
11    }
12
13    egress {
14        from_port   = 0
15        to_port     = 0
16        protocol    = "-1"
17        cidr_blocks = ["0.0.0.0/0"]
18    }
19 }

```

- project4_part2_security_rules_summary.png

Security Group	Direction	Protocol	Port(s)	Source/Destination
Bastion SG	Ingress	TCP	22 (SSH)	Admin IP (4.240.18.227/32)
	Egress	All	All	0.0.0.0/0
Web SG	Ingress	TCP	80 (HTTP)	0.0.0.0/0 (anywhere)
	Ingress	TCP	22 (SSH)	Bastion SG
	Egress	All	All	0.0.0.0/0
App SG	Ingress	TCP	8080 (App Port)	Web SG
	Ingress	TCP	22 (SSH)	Bastion SG
	Egress	All	All	0.0.0.0/0

5.3 Compute Resources

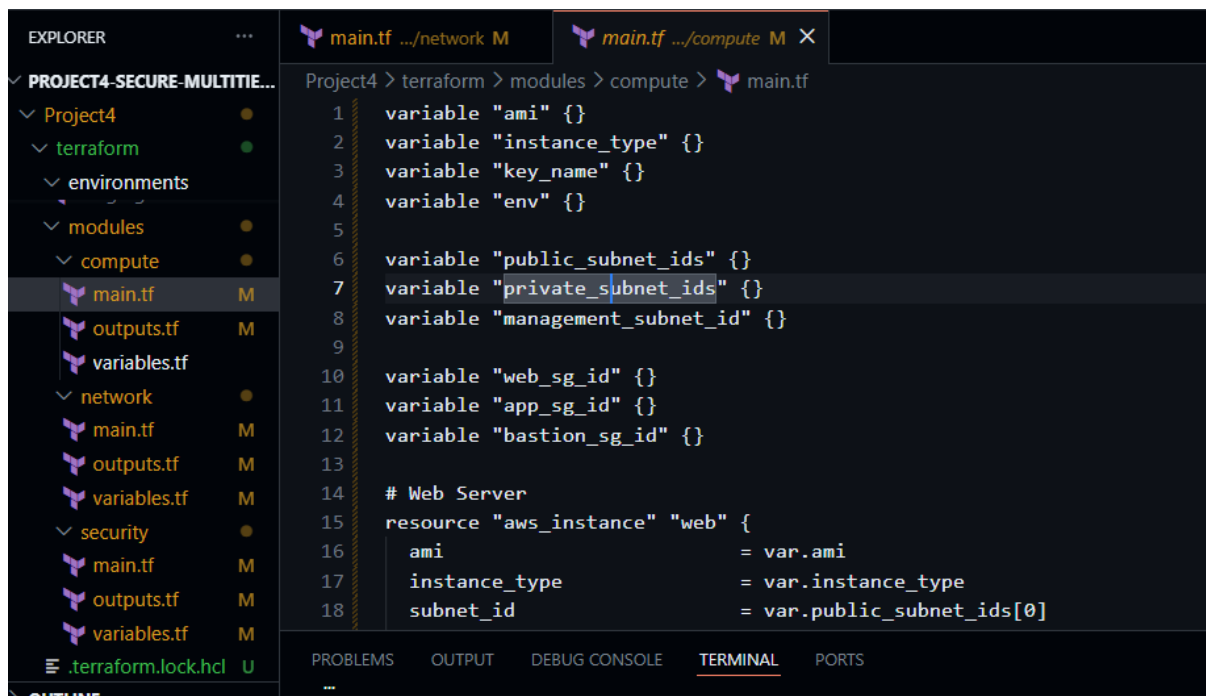
Terraform provisions:

- **Web servers** in public subnets
- **App servers** in private subnets
- **Bastion host** in management subnet

All resources are tagged with Project=Project4 and Tier=management/web/app for dynamic inventory and traceability.

Screenshots:

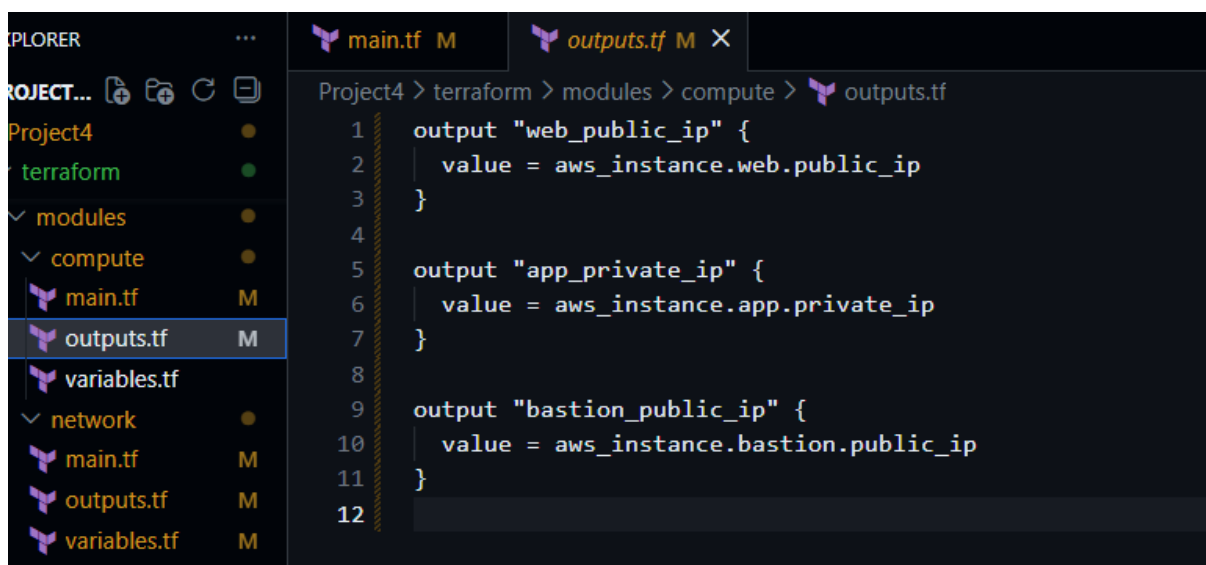
- project4_part2_compute_main.png



The screenshot shows the Visual Studio Code interface with the Explorer panel on the left and the main editor on the right. The Explorer panel shows a project structure with folders for Project4, terraform, environments, modules, and compute. The compute folder is expanded, showing main.tf, outputs.tf, and variables.tf. The main.tf file is selected and its content is displayed in the main editor. The code defines variables for ami, instance_type, key_name, env, public_subnet_ids, private_subnet_ids, management_subnet_id, web_sg_id, app_sg_id, and bastion_sg_id. It also defines a resource 'aws_instance' named 'web' with attributes ami, instance_type, and subnet_id.

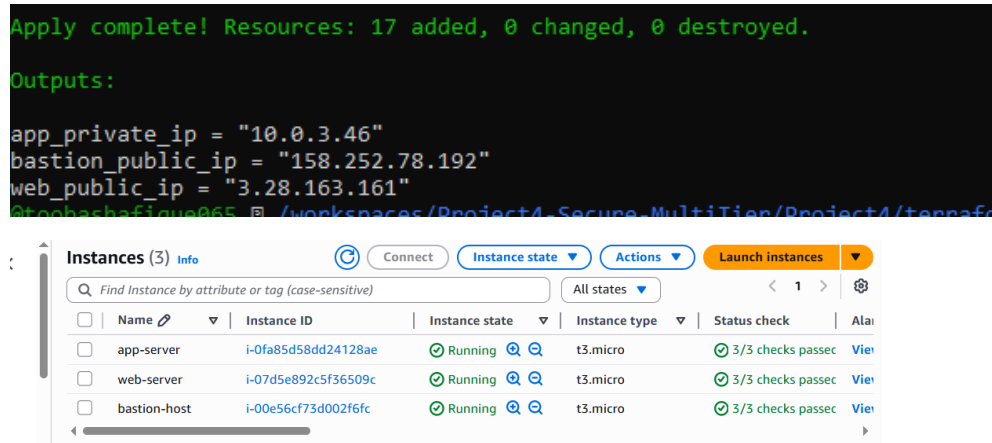
```
Project4 > terraform > modules > compute > main.tf
1  variable "ami" {}
2  variable "instance_type" {}
3  variable "key_name" {}
4  variable "env" {}
5
6  variable "public_subnet_ids" {}
7  variable "private_subnet_ids" {}
8  variable "management_subnet_id" {}
9
10 variable "web_sg_id" {}
11 variable "app_sg_id" {}
12 variable "bastion_sg_id" {}
13
14 # Web Server
15 resource "aws_instance" "web" {
16     ami           = var.ami
17     instance_type = var.instance_type
18     subnet_id     = var.public_subnet_ids[0]
```

- project4_part2_compute_outputs.png



The screenshot shows the Visual Studio Code interface with the Explorer panel on the left and the main editor on the right. The Explorer panel shows the same project structure as the previous screenshot, but the compute folder is expanded to show outputs.tf, which is selected. The main editor displays the content of outputs.tf, which defines three outputs: 'web_public_ip', 'app_private_ip', and 'bastion_public_ip', each with a value derived from the 'aws_instance' resource.

```
Project4 > terraform > modules > compute > outputs.tf
1  output "web_public_ip" {
2      value = aws_instance.web.public_ip
3  }
4
5  output "app_private_ip" {
6      value = aws_instance.app.private_ip
7  }
8
9  output "bastion_public_ip" {
10     value = aws_instance.bastion.public_ip
11 }
12
```



6. Part 3 — Root Terraform & Environments

6.1: The root module includes:

- main.tf → AWS provider and module instantiation
- variables.tf → input variables
- outputs.tf → public/private IPs
- backend.tf → remote state configuration

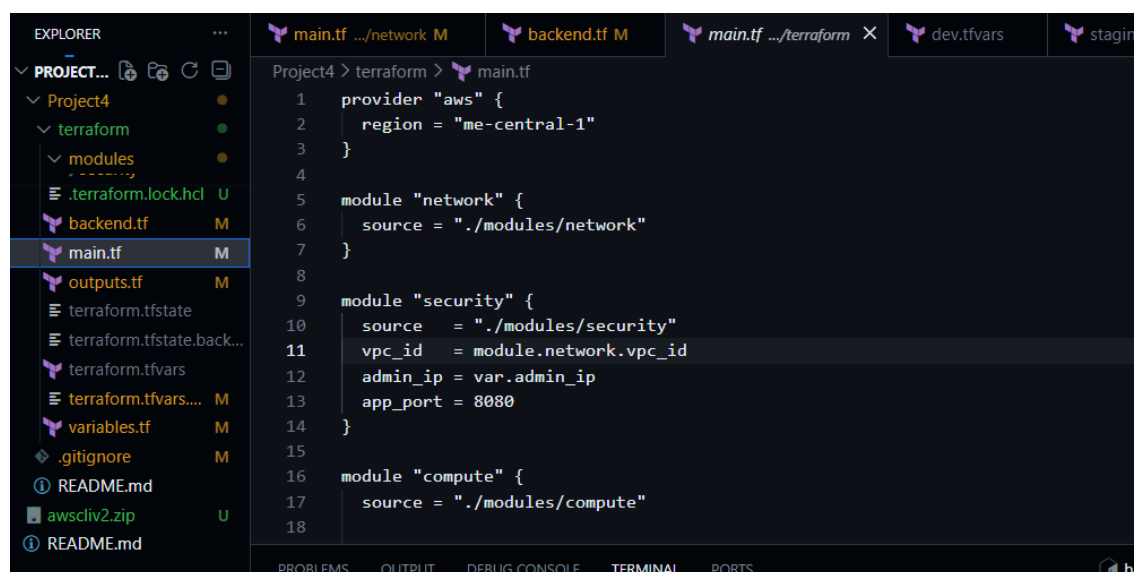
Modules are wired together: VPC → Subnets → Security Groups → EC2 instances.

Outputs include:

- Bastion public IP
- Web public endpoint
- App private IPs

Deliverables:

- Screenshot: project4_part3_main_tf.png



- Screenshot: project4_part3_outputs_tf.png

```

1  # Public IP for Bastion Host (SSH)
2  output "bastion_public_ip" {
3    value = module.compute.bastion_public_ip
4  }
5
6  # Public IPs of Web Servers
7  output "web_public_ips" {
8    value = [module.compute.web_public_ip]
9  }
10
11 # Private IPs of App Servers
12 output "app_private_ips" {
13   value = [module.compute.app_private_ip]
14 }
15
16 # Optional: For reference
17 output "vpc_id" {
18   value = module.network.vpc_id
19 }

```

6.2 Environment-Specific Variables Files:

Environment files (dev.tfvars, staging.tfvars, production.tfvars) define:

- CIDR blocks
- Admin IPs
- Instance types and counts
- Consistent region: me-central-1

This enables flexible deployments across multiple environments.

Deliverables:

- Screenshot: project4_part3_dev_tfvars.png

```

1  region      = "me-central-1"
2  ami         = "ami-06a8303596865117b"
3  instance_type = "t3.micro"
4  key_name    = "project4-key"
5  env         = "dev"
6  admin_ip    = "4.240.18.227/32"
7  app_port    = 8080
8

```

- Screenshot: project4_part3_staging_tfvars.png

```
main.tf M  backend.tf M  dev.tfvars  staging.tfvars X  ≡
Project4 > terraform > environments > staging.tfvars
1  region      = "me-central-1"
2  ami          = "ami-06a8303596865117b"
3  instance_type = "t3.small"
4  key_name     = "project4-key"
5  env          = "staging"
6  admin_ip     = "4.240.18.227/32"
7  app_port     = 8080
8
```

- Screenshot: project4_part3_production_tfvars.png

```
main.tf M  backend.tf M  dev.tfvars  staging.tfvars  production.tfvars X  ≡
Project4 > terraform > environments > production.tfvars
1  region      = "me-central-1"
2  ami          = "ami-06a8303596865117b"
3  instance_type = "t3.medium"
4  key_name     = "project4-key"
5  env          = "production"
6  admin_ip     = "4.240.18.227/32"
7  app_port     = 8080
8
```

- Screenshot: project4_part3_tfvars_example.png

```
Project4 > terraform > ≡ terraform.tfvars.example
1  region      = "me-central-1"
2  ami          = "<AMI_ID>"
3  instance_type = "<INSTANCE_TYPE>"
4  key_name     = "<KEY_NAME>"
5  env          = "<ENV_NAME>"
6  admin_ip     = "<ADMIN_IP>/32"
7  app_port     = 8080
8
```

After running:

```
terraform init
```

```
terraform plan -var-file=environments/dev.tfvars
```

```
terraform apply -var-file=environments/dev.tfvars -auto-approve
```

Outputs:

```
app_private_ips = [
    "10.0.3.46",
]
bastion_public_ip = "158.252.78.192"
management_subnet_id = "subnet-02c6f296963375734"
private_subnet_ids = [
    "subnet-0c293c44ac6ebbac1",
    "subnet-04869e1849bab6d56",
]
public_subnet_ids = [
    "subnet-08737804e5d700d64",
    "subnet-09325ebb0d196b1ea",
]
vpc_id = "vpc-007866a978d8887f7"
web_public_ips = [
    "3.28.163.161",
]
```

7. Part 4 — Ansible Implementation

7.1 Dynamic Inventory & Global Configuration

Ansible uses AWS EC2 dynamic inventory with tag-based grouping:

- Hosts are grouped by Tier tag
- ansible.cfg is configured for SSH key usage and inventory path

Deliverables:

- Screenshot: project4_part4_ansible_inventory_dev.png

```
Project4 > ansible > inventory > ! dev_aws_ec2.yml
1  plugin: aws_ec2
2  regions:
3  | - me-central-1
4  filters:
5  | tag:Project: "Project4"
6  | tag:Environment: "dev"
7  keyed_groups:
8  | - key: tags.Tier
9  |   prefix: tier
10 hostnames:
11 | - private-ip-address
12 compose:
13 | ansible_host: private_ip_address
14
```

- Screenshot: project4_part4_ansible_cfg.png

```
ansible.cfg M X
Project4 > ansible > ansible.cfg
1 [defaults]
2 result_format = yaml
3 inventory = inventory/dev_aws_ec2.yml
4 host_key_checking = False
5 retry_files_enabled = False
6 gathering = smart
7 stdout_callback = default
8 roles_path = roles
9 remote_user = ec2-user
10 private_key_file = ~/.ssh/project4-key.pem
11
12 [privilege_escalation]
13 become = True
14 become_method = sudo
15
16 [ssh_connection]
17 pipelining = True
18 ssh_args = -o ProxyCommand="ssh -i ~/.ssh/project4-key.pem -o StrictHostKeyChecking=no -W %h:
19
```

- Screenshot: project4_part4_group_vars_all.png

```
Project4 > ansible > group_vars > ! all.yml
1 # SSH Hardening
2 permit_root_login: "no"
3 password_authentication: "no"
4 admin_user: "deployer"
5
6 # App Configuration
7 app_port: 8080
8 nginx_upstream_port: 8080
9
```

- Inventory graph

```
[DEPRECATION WARNING]: Passing 'disable_lookups' to
@all:
|--@ungrouped:
|--@aws_ec2:
|  |--10.0.5.162
|  |--10.0.1.54
|  |--10.0.3.46
|--@tier_management:
|  |--10.0.5.162
|--@tier_web:
|  |--10.0.1.54
|--@tier_app:
|  |--10.0.3.46
```

7.2 Server Hardening

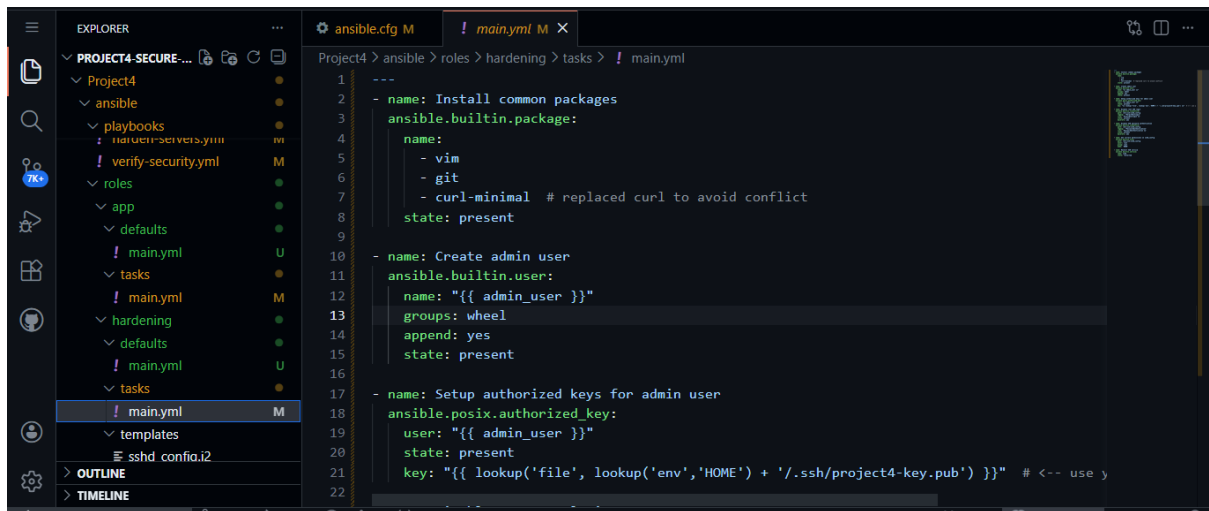
The hardening role performs:

- Creation of non-root deployer user

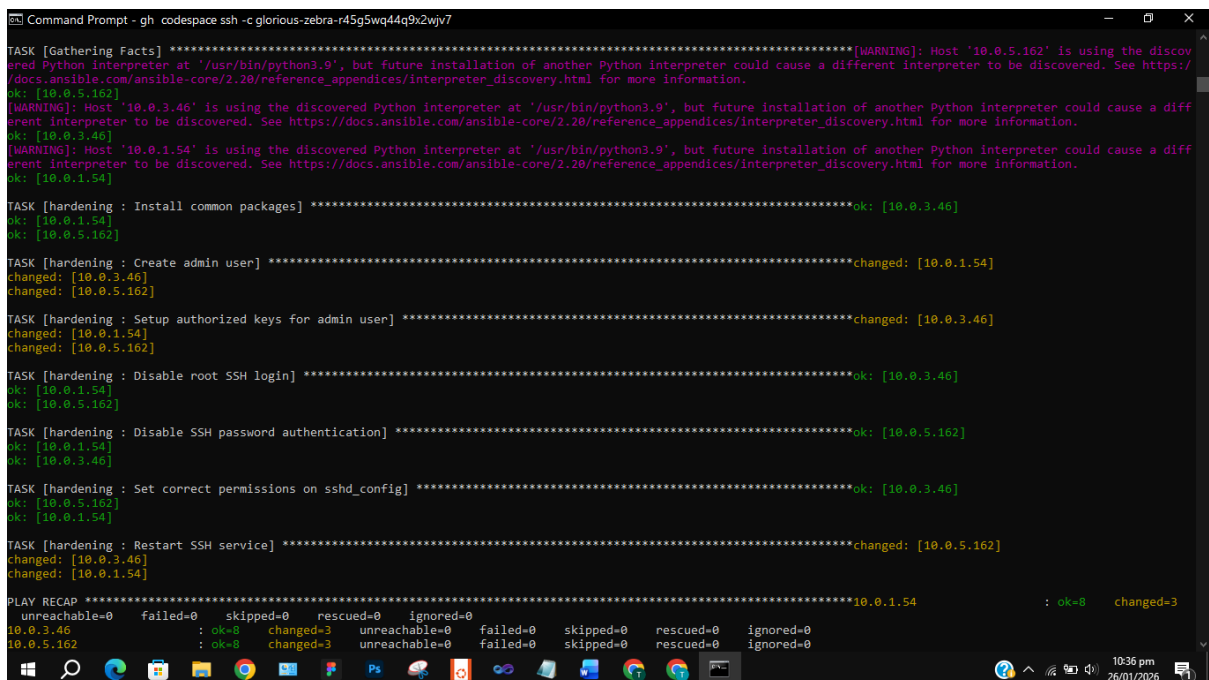
- Disabling root login
- Enforcing SSH key-only access
- Fixing permissions on .ssh and sshd_config

Deliverables:

- Screenshot: project4_part4_hardening_role_main.png



- Screenshot: project4_part4_harden_execution.png



- project4_part5_sshd_config_evidence.png


```
deployer@ip-10-0-3-46:~$
Warning: Permanently added the IP address 10.0.5.162 to the list of known hosts.
Warning: Permanently added the IP address 10.0.1.54 to the list of known hosts.
Warning: Permanently added the IP address 10.0.3.46 to the list of known hosts.
TASK [Check SSHD config for root login and password auth] *****
changed: [10.0.5.162]
changed: [10.0.1.54]
changed: [10.0.3.46]
TASK [debug] *****
ok: [10.0.5.162] => {
  "msg": "10.0.5.162 -> ['PermitRootLogin prohibit-password', '# Explicitly disable PasswordAuthentication. By presetting it, we', 'PasswordAuthentication no', '# PasswordAuthentication. Depending on your PAM configuration,', '# the setting of \"PermitRootLogin without-password\".', '# PAM authentication, then enable this but set PasswordAuthentication']"
}
ok: [10.0.1.54] => {
  "msg": "10.0.1.54 -> ['PermitRootLogin prohibit-password', '# Explicitly disable PasswordAuthentication. By presetting it, we', 'PasswordAuthentication no', '# PasswordAuthentication. Depending on your PAM configuration,', '# the setting of \"PermitRootLogin without-password\".', '# PAM authentication, then enable this but set PasswordAuthentication']"
}
ok: [10.0.3.46] => {
  "msg": "10.0.3.46 -> ['PermitRootLogin prohibit-password', '# Explicitly disable PasswordAuthentication. By presetting it, we', 'PasswordAuthentication no', '# PasswordAuthentication. Depending on your PAM configuration,', '# the setting of \"PermitRootLogin without-password\".', '# PAM authentication, then enable this but set PasswordAuthentication']"
}
TASK [Check if server has public IP] *****
ok: [10.0.5.162] => {
  "msg": "10.0.5.162 public IP: 10.0.5.162"
}
ok: [10.0.1.54] => {
  "msg": "10.0.1.54 public IP: 10.0.1.54"
}
ok: [10.0.3.46] => {
  "msg": "10.0.3.46 public IP: 10.0.3.46"
}
PLAY RECAP *****
10.0.1.54 : ok=4 changed=1 unreachable=0 failed=0 skipped=0 rescued=0 ignored=0
10.0.3.46 : ok=4 changed=1 unreachable=0 failed=0 skipped=0 rescued=0 ignored=0
10.0.5.162 : ok=4 changed=1 unreachable=0 failed=0 skipped=0 rescued=0 ignored=0
```

8.2 Security Compliance Checklist

Checklist confirms:

- Root login is disabled
- SSH key-only access is enforced
- App tier has no public IPs
- Logs are successfully collected

Deliverables:

- Screenshot: project4_part5_security_checklist_doc.png

```
Project4 > docs > security-checklist.md > abc # Security Checklist
1  # Security Checklist
2
3  ✓ VPC configured with public and private subnets
4  ✓ Application servers have no public IPs
5  ✓ Bastion host used as single SSH entry point
6  ✓ SSH access restricted to administrator IPs
7  ✓ Security groups follow least-privilege rules
8  ✓ Only required ports are open (80, 22, 8080)
9  ✓ Nginx reverse proxy used for traffic forwarding
10 ✓ Application tier isolated from Internet access
11 ✓ SSH password authentication disabled
12 ✓ Key-based authentication enforced
13 |
```

9. Part 6 — Challenges & Solutions

1. Terraform Deployment Issues

- **Challenge:** Initial Terraform runs failed due to AMI/region mismatches and unsupported instance types. Some resources didn't provision correctly because of missing tags required for Ansible dynamic inventory.
- **Solution:** Verified AMI IDs and region compatibility, updated Terraform variables, and ensured all EC2 instances had consistent Project and Tier tags. Re-ran terraform apply until infrastructure deployed cleanly.

2. SSH Access & Hardening Challenges

- **Challenge:**
 - Direct SSH to app tier failed (no public IPs).
 - deployer user couldn't read /etc/ssh/sshd_config without sudo.
 - Ansible hardening role initially misconfigured group membership (sudo vs wheel).
- **Solution:**
 - Used bastion host with ProxyCommand for secure SSH access.
 - Enabled become: true in Ansible playbooks to check SSHD config without needing a password.
 - Corrected group membership and variable scoping in Ansible roles to ensure deployer had sudo privileges.

3. Nginx Proxy / Networking Issues

- **Challenge:**
 - Nginx initially returned 502 Bad Gateway because the Flask app wasn't running or listening on the correct port.
 - Health endpoint misconfigured in nginx.conf.
- **Solution:**
 - Verified Flask app with curl localhost:8080.
 - Updated Nginx upstream block to point to app tier private IPs on port 8080.
 - Added /health route in both Flask app and Nginx config.
 - Restarted services and confirmed proxy worked with curl http://<web-tier-public-ip>.

4. Log Collection Issues

- **Challenge:**
 - Ansible fetch task failed for /var/log/secure because Amazon Linux 2023 uses journalctl instead of flat log files.
 - Logs initially saved outside the project folder due to wrong dest path (../ instead of ../).
- **Solution:**
 - Adjusted playbook to store logs under docs/logs/.

- Documented the OS difference in README (SSH logs unavailable, but Nginx logs collected successfully).
- Verified with tree docs/logs/ showing access.log and error.log.

10. Part 7 — Conclusion & Future Improvements

Final Outcome

- Successfully deployed a **secure multi-tier architecture** on AWS using Terraform and Ansible.
- Implemented **hardening measures** (non-root user, SSH key-only access, disabled root login).
- Configured **Nginx reverse proxy** to route traffic from web tier to app tier.
- Verified compliance with security checklist and collected logs for evidence.

Security Improvements Achieved

- **Root login disabled** → prevents unauthorized root access.
- **Password authentication disabled** → enforces key-only SSH.
- **App tier isolated** → no public IPs, accessible only via bastion.
- **Centralized logging** → Nginx logs collected for audit and evidence.
- **Configuration automation** → reproducible and consistent security setup via Ansible.

Future Enhancements

- **Monitoring & Alerting** → integrate CloudWatch or Prometheus/Grafana for real-time metrics and alerts.
- **SSL/TLS Certificates** → secure Nginx reverse proxy with HTTPS using Let's Encrypt.
- **Scaling & Load Balancing** → add AWS ALB/ELB to distribute traffic across multiple app servers.
- **CI/CD Integration** → automate deployment pipelines for faster updates.
- **Centralized Log Management** → forward logs to ELK stack (Elasticsearch, Logstash, Kibana) for analysis.

11. Appendices

Appendix A — Code Snippets

A.1 Terraform Tagging Evidence

```
tags = {  
  Project    = "Project4"  
  Environment = "dev"  
  Tier       = "app"  
}
```


1–2 lines explanation:

This Terraform snippet shows consistent resource tagging used for environment isolation, inventory discovery, and tier identification.

A.2 Ansible SSH Hardening Task

```
- name: Disable root login
```

```
  lineinfile:
```

```
    path: /etc/ssh/sshd_config
```

```
    regexp: '^PermitRootLogin'
```

```
    line: 'PermitRootLogin no'
```

```
- name: Disable password authentication
```

```
  lineinfile:
```

```
    path: /etc/ssh/sshd_config
```

```
    regexp: '^PasswordAuthentication'
```

```
    line: 'PasswordAuthentication no'
```

Explanation:

These Ansible tasks enforce SSH hardening by disabling root login and password-based authentication.

A.3 Nginx Reverse Proxy Upstream

```
upstream app_backend {
```

```
  server 10.0.3.46:8080;
```

```
}
```

Explanation:

This configuration enables Nginx to forward traffic from the web tier to the private application tier.

Appendix B — Logs

Shows collected logs from my system:

- docs/logs/10.0.1.54/nginx/access.log.

```
Project4 > docs > logs > 10.0.1.54 > nginx > 10.0.1.54 > var > log > nginx > access.log
1  4.240.39.198 - - [27/Jan/2026:01:20:17 +0000] "GET / HTTP/1.1" 502 157 "-" "curl/8.5.0"
2  4.240.39.198 - - [27/Jan/2026:01:41:18 +0000] "GET / HTTP/1.1" 200 20 "-" "curl/8.5.0"
3
```

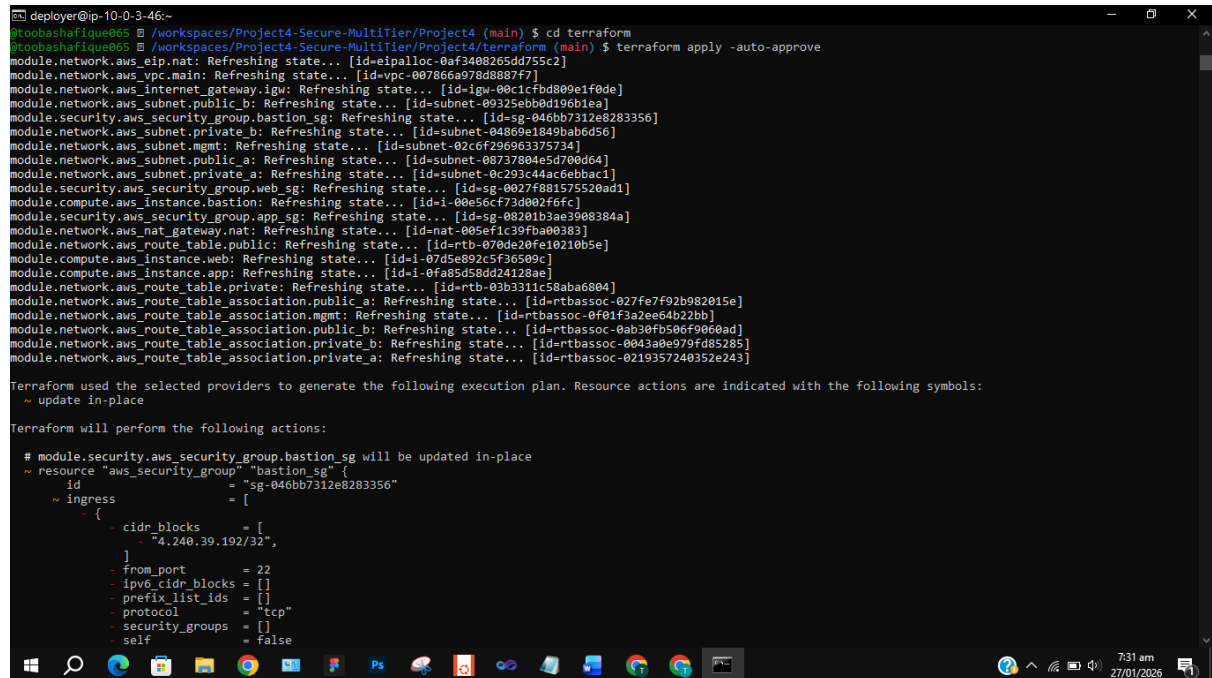
- docs/logs/10.0.1.54/nginx/error.log.

```
Project4 > docs > logs > 10.0.1.54 > nginx > 10.0.1.54 > var > log > nginx > error.log
1 2026/01/27 01:20:17 [error] 119265#119265: *1 connect() failed (111: Connection refused) while conn
2
```

Appendix C — Screenshots

Insert all screenshots you've taken throughout the project:

- project4_part3_terraform_apply.png



```
deployer@ip-10-0-3-46:~
$ cd /workspaces/Project4-Secure-MultiTier/Project4
$ cd terraform
$ terraform apply -auto-approve
module.network.aws_vpc.main: Refreshing state... [id=vpc-007866a978d8887f7]
module.network.aws_internet_gateway.igw: Refreshing state... [id=igw-00c1cfbd809e1f0de]
module.network.aws_subnet.public_b: Refreshing state... [id=subnet-09325ebb0d196b1ea]
module.security.aws_security_group.bastion_sg: Refreshing state... [id=sg-046bb7312e8283356]
module.network.aws_subnet.private_b: Refreshing state... [id=subnet-04869e1849bab6d56]
module.network.aws_subnet.mgmt: Refreshing state... [id=subnet-02c6f296963375734]
module.network.aws_subnet.public_a: Refreshing state... [id=subnet-08737804e5d700d64]
module.network.aws_subnet.private_a: Refreshing state... [id=subnet-0c293c44ac6ebb1]
module.security.aws_security_group.web_sg: Refreshing state... [id=sg-0027f38197520ad1]
module.compute.aws_instance.bastion: Refreshing state... [id=i-00e56cf73d002f6fc]
module.security.aws_security_group.app_sg: Refreshing state... [id=sg-08201b3ae3908384a]
module.network.aws_nat_gateway.nat: Refreshing state... [id=nat-005ef1c39fba00383]
module.network.aws_route_table.public: Refreshing state... [id=rtb-070de20fe10210b5e]
module.compute.aws_instance.web: Refreshing state... [id=i-07d5e892c5f36509c]
module.compute.aws_instance.app: Refreshing state... [id=i-0fa85d58d24128ae]
module.network.aws_route_table.private: Refreshing state... [id=rtb-03b3311c58aba6804]
module.network.aws_route_table_association.public_a: Refreshing state... [id=rtbassoc-027fe7f92b082015e]
module.network.aws_route_table_association.mgmt: Refreshing state... [id=rtbassoc-0f01f3a2e64b22bb]
module.network.aws_route_table_association.public_b: Refreshing state... [id=rtbassoc-0ab30fb506f9060ad]
module.network.aws_route_table_association.private_b: Refreshing state... [id=rtbassoc-0043a0e979fd85285]
module.network.aws_route_table_association.private_a: Refreshing state... [id=rtbassoc-0219357240352e243]

Terraform used the selected providers to generate the following execution plan. Resource actions are indicated with the following symbols:
~ update in-place

Terraform will perform the following actions:

# module.security.aws_security_group.bastion_sg will be updated in-place
~ resource "aws_security_group" "bastion_sg" {
  id          = "sg-046bb7312e8283356"
  ingress    = [
    {
      cidr_blocks = [
        "4.240.39.192/32",
      ]
      from_port   = 22
      ipv6_cidr_blocks = []
      prefix_list_ids = []
      protocol    = "tcp"
      security_groups = []
      self        = false
    }
  ]
}
```

- project4_part3_terraform_output.png

```
Outputs:

app_private_ips = [
  "10.0.3.46",
]
bastion_public_ip = "158.252.78.192"
management_subnet_id = "subnet-02c6f296963375734"
private_subnet_ids = [
  "subnet-0c293c44ac6ebb1",
  "subnet-04869e1849bab6d56",
]
public_subnet_ids = [
  "subnet-08737804e5d700d64",
  "subnet-09325ebb0d196b1ea",
]
vpc_id = "vpc-007866a978d8887f7"
web_public_ips = [
  "3.28.163.161",
]
```

- project4_part4_collect_logs_playbook.png → playbook run for log collection.

```

deployer@ip-10-0-3-46:~
logout
Connection to 10.0.3.46 closed.
@toobashafique065 /workspaces/Project4-Secure-MultiTier/Project4/ansible (main) $ ansible-playbook -i inventory/dev_aws_ec2.yml playbooks/verify-security.yml
[WARNING]: Deprecation warnings can be disabled by setting 'deprecation_warnings=False' in ansible.cfg.
[DEPRECATION WARNING]: Importing 'to_text' from 'ansible.module_utils._text' is deprecated. This feature will be removed from ansible-core version 2.24. Use ansible.module_utils.common.text.converters instead.
[DEPRECATION WARNING]: Importing 'to_native' from 'ansible.module_utils._text' is deprecated. This feature will be removed from ansible-core version 2.24. Use ansible.module_utils.common.text.converters instead.
[DEPRECATION WARNING]: Passing 'disable_lookups' to 'template' is deprecated. This feature will be removed from ansible-core version 2.23.

PLAY [Verify Security Settings] *****
[WARNING]: Found variable using reserved name 'tags'.
Origin: <unknown>

tags

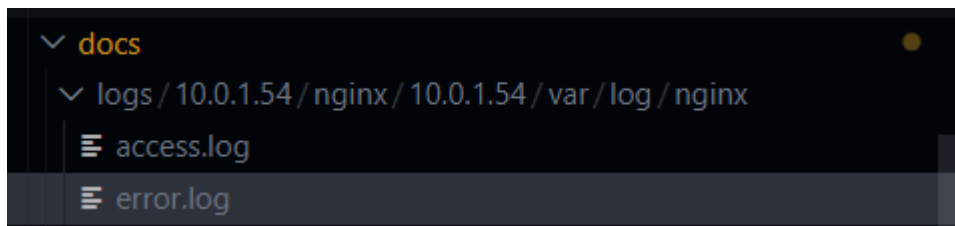
TASK [Gathering Facts] *****
[WARNING]: Host '10.0.3.46' is using the discovered Python interpreter at '/usr/bin/python3.9', but future installation of another Python interpreter could cause a different interpreter to be discovered. See https://docs.ansible.com/ansible-core/2.20/reference_appendices/interpreter_discovery.html for more information.
ok: [10.0.3.46]
[WARNING]: Host '10.0.5.162' is using the discovered Python interpreter at '/usr/bin/python3.9', but future installation of another Python interpreter could cause a different interpreter to be discovered. See https://docs.ansible.com/ansible-core/2.20/reference_appendices/interpreter_discovery.html for more information.
ok: [10.0.5.162]
[WARNING]: Host '10.0.1.54' is using the discovered Python interpreter at '/usr/bin/python3.9', but future installation of another Python interpreter could cause a different interpreter to be discovered. See https://docs.ansible.com/ansible-core/2.20/reference_appendices/interpreter_discovery.html for more information.
ok: [10.0.1.54]

TASK [Check SSHD config for root login and password auth] *****
changed: [10.0.3.46]
changed: [10.0.1.54]
changed: [10.0.5.162]

TASK [debug] *****
ok: [10.0.5.162] => {
  "msg": "10.0.5.162 -> ['PermitRootLogin prohibit-password', '# Explicitly disable PasswordAuthentication. By presetting it, we', 'PasswordAuthentication no', 'PasswordAuthentication. Depending on your PAM configuration,', '# the setting of \"PermitRootLogin without-password\".', '# PAM authentication, then enable this but set PasswordAuthentication']"
}
ok: [10.0.1.54] => {
  "msg": "10.0.1.54 -> ['PermitRootLogin prohibit-password', '# Explicitly disable PasswordAuthentication. By presetting it, we', 'PasswordAuthentication no', 'PasswordAuthentication. Depending on your PAM configuration,', '# the setting of \"PermitRootLogin without-password\".', '# PAM authentication, then enable this but set PasswordAuthentication']"
}

```

- project4_part4_logs_collected_tree.png → tree view of collected logs.



```

Project4 > docs > logs > 10.0.1.54 > nginx > 10.0.1.54 > var > log > nginx > access.log
1 4.240.39.198 - - [27/Jan/2026:01:20:17 +0000] "GET / HTTP/1.1" 502 157 "-" "curl/8.5.0"
2 4.240.39.198 - - [27/Jan/2026:01:41:18 +0000] "GET / HTTP/1.1" 200 20 "-" "curl/8.5.0"
3

```

```

Project4 > docs > logs > 10.0.1.54 > nginx > 10.0.1.54 > var > log > nginx > error.log
1 2026/01/27 01:20:17 [error] 119265#119265: *1 connect() failed (111: Connection refused) while conn
2

```

- project4_part5_bastion_ssh_flow.png → failed direct SSH, successful via bastion.

```

@toobashafique065 /workspaces/Project4-Secure-MultiTier/Project4/ansible (main) $ ssh -i ~/.ssh/project4-key.pem ec2-user@10.0.3.46
ssh: connect to host 10.0.3.46 port 22: Connection timed out
@toobashafique065 /workspaces/Project4-Secure-MultiTier/Project4/ansible (main) $

```

- project4_part4_flask_local_test.png
- Verification was performed using both ec2-user and deployer accounts. Since services run system-wide, both accounts produced the same output.


```
ec2 deployer@ip-10-0-3-46:~
$ ansible-playbook -i inventory project4_part5_security_checklist.yml
WARNING: Dependent interpreter to be discovered. See https://docs.ansible.com/ansible-core/2.20/reference_appendices/interpreter_discovery.html for more information.
ok: [10.0.5.162]
[WARNING]: Host '10.0.1.54' is using the discovered Python interpreter at '/usr/bin/python3.9', but future installation of another Python interpreter could cause a different interpreter to be discovered. See https://docs.ansible.com/ansible-core/2.20/reference_appendices/interpreter_discovery.html for more information.
ok: [10.0.1.54]

TASK [Check SSHD config for root login and password auth] *****
changed: [10.0.3.46]
changed: [10.0.1.54]
changed: [10.0.5.162]

TASK [debug] *****
ok: [10.0.5.162] => {
  "msg": "10.0.5.162 -> ['PermitRootLogin prohibit-password', '# Explicitly disable PasswordAuthentication. By presetting it, we', 'PasswordAuthentication no', 'PasswordAuthentication. Depending on your PAM configuration,', '# the setting of \"PermitRootLogin without-password\".', '# PAM authentication, then enable this but set PasswordAuthentication']"
}
ok: [10.0.1.54] => {
  "msg": "10.0.1.54 -> ['PermitRootLogin prohibit-password', '# Explicitly disable PasswordAuthentication. By presetting it, we', 'PasswordAuthentication no', 'PasswordAuthentication. Depending on your PAM configuration,', '# the setting of \"PermitRootLogin without-password\".', '# PAM authentication, then enable this but set PasswordAuthentication']"
}
ok: [10.0.3.46] => {
  "msg": "10.0.3.46 -> ['PermitRootLogin prohibit-password', '# Explicitly disable PasswordAuthentication. By presetting it, we', 'PasswordAuthentication no', 'PasswordAuthentication. Depending on your PAM configuration,', '# the setting of \"PermitRootLogin without-password\".', '# PAM authentication, then enable this but set PasswordAuthentication']"
}

TASK [Check if server has public IP] *****
ok: [10.0.5.162] => {
  "msg": "10.0.5.162 public IP: 10.0.5.162"
}
ok: [10.0.1.54] => {
  "msg": "10.0.1.54 public IP: 10.0.1.54"
}
ok: [10.0.3.46] => {
  "msg": "10.0.3.46 public IP: 10.0.3.46"
}

PLAY RECAP *****
10.0.1.54      : ok=4    changed=1    unreachable=0    failed=0    skipped=0    rescued=0    ignored=0
10.0.3.46      : ok=4    changed=1    unreachable=0    failed=0    skipped=0    rescued=0    ignored=0
10.0.5.162     : ok=4    changed=1    unreachable=0    failed=0    skipped=0    rescued=0    ignored=0
```

- project4_part5_security_checklist_doc.png → compliance checklist ticked off.

```
Project4 > docs > security-checklist.md > abc # Security Checklist
1  # Security Checklist
2
3  ✓ VPC configured with public and private subnets
4  ✓ Application servers have no public IPs
5  ✓ Bastion host used as single SSH entry point
6  ✓ SSH access restricted to administrator IPs
7  ✓ Security groups follow least-privilege rules
8  ✓ Only required ports are open (80, 22, 8080)
9  ✓ Nginx reverse proxy used for traffic forwarding
10 ✓ Application tier isolated from Internet access
11 ✓ SSH password authentication disabled
12 ✓ Key-based authentication enforced
13 |
```

Appendix D — Troubleshooting Notes

SSH Connectivity Issues

```
ssh -i ~/.ssh/project4-key.pem ec2-user@10.0.3.46
```

Failed as expected due to private subnet and no public IP.

```
ssh -i ~/.ssh/project4-key.pem \
```

```
-o ProxyCommand="ssh -i ~/.ssh/project4-key.pem -W %h:%p ec2-user@158.252.78.192" \
```

```
deployer@10.0.3.46
```

Successful SSH access via bastion host.

Reverse Proxy Verification

```
curl http://3.28.163.161/
```

Confirmed Nginx reverse proxy forwarded traffic to the application tier.

Security Verification

```
ansible-playbook -i inventory/dev_aws_ec2.yml playbooks/verify-security.yml
```

Verified SSH hardening and security configuration across all tiers.